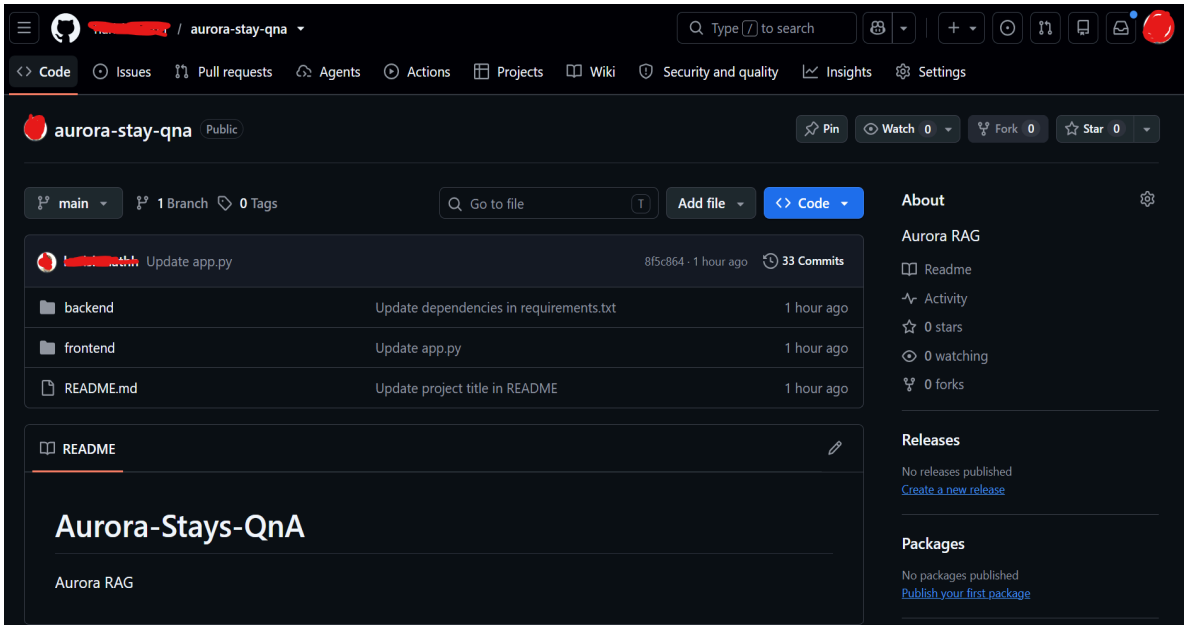
Deploying a RAG Application Using GitHub Codespaces

This document provides a detailed, step-by-step walkthrough for deploying a RAG (Retrieval-Augmented Generation) application using GitHub Codespaces. The application consists of a backend API and a Streamlit-based frontend. Both components are containerized using Docker, built inside the Codespace environment, and then run as containers to serve the application.

Section 1: Opening the GitHub Repository

Step 1: Open the GitHub repository that contains the project code. The repository should have a backend folder, a frontend folder, and a README.md file. Verify that all the required project files are present before moving ahead.



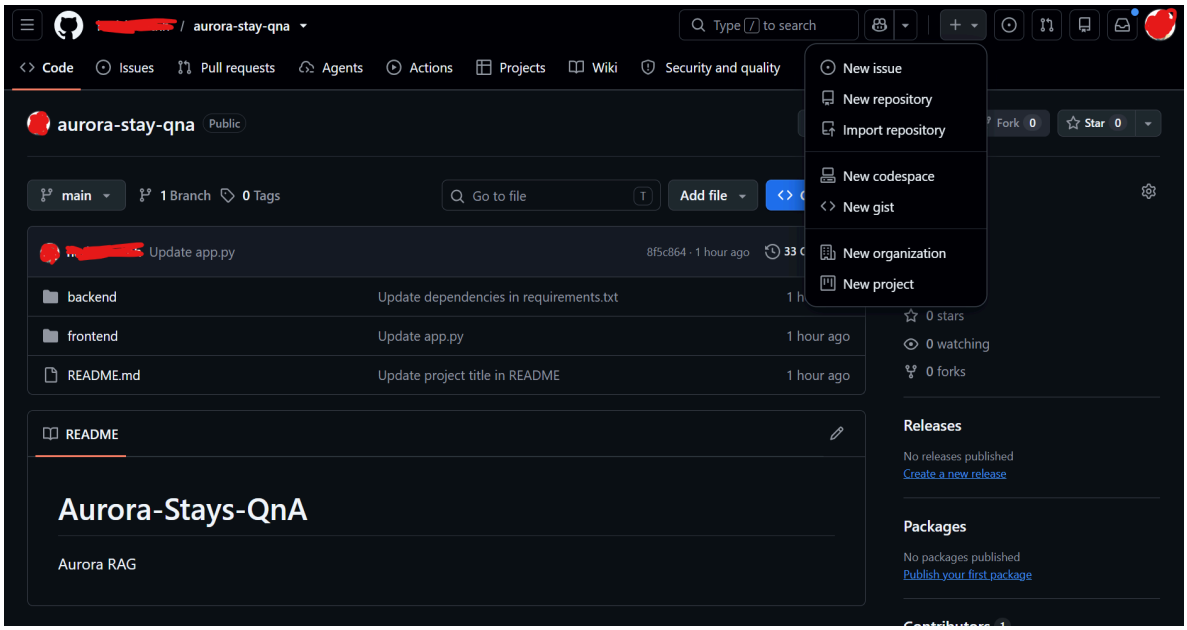
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces

Section 2: Creating a New GitHub Codespace

Step 2: On the repository page, click the "+" icon located in the top-right area of the GitHub navigation bar. From the dropdown menu that appears, select New codespace.

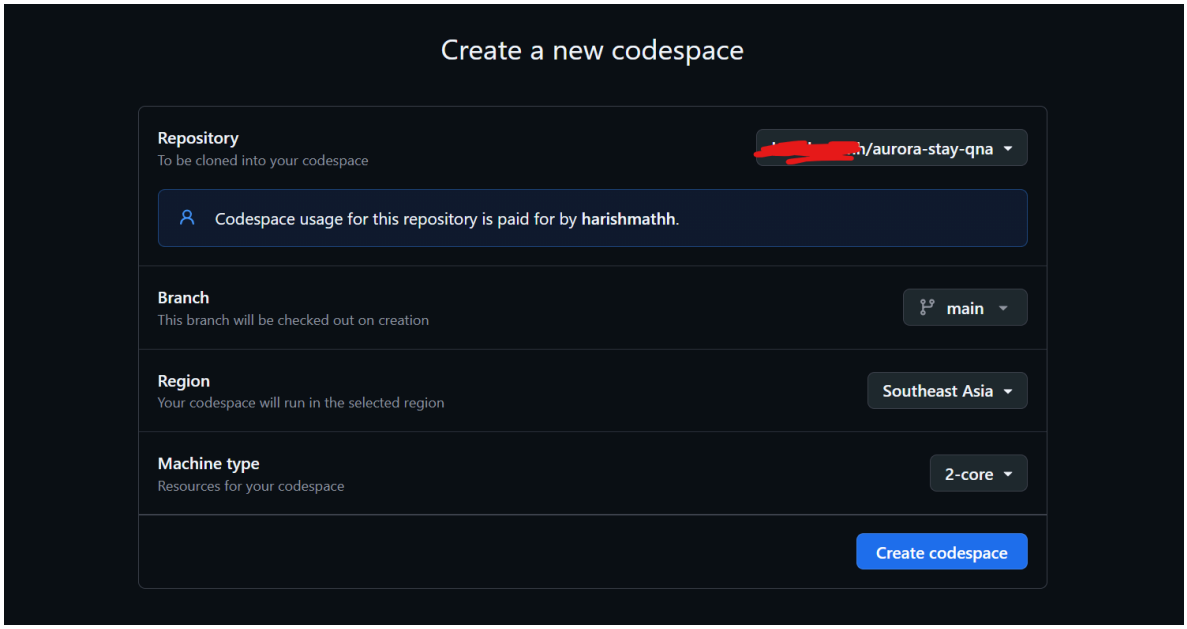


Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces

Step 3: A configuration page will appear titled "Create a new codespace". Here, confirm that the correct repository is selected. Choose the branch (main), select a region (for example, Southeast Asia), and pick a machine type (2-core is sufficient for this project). Once everything looks correct, click the Create codespace button.

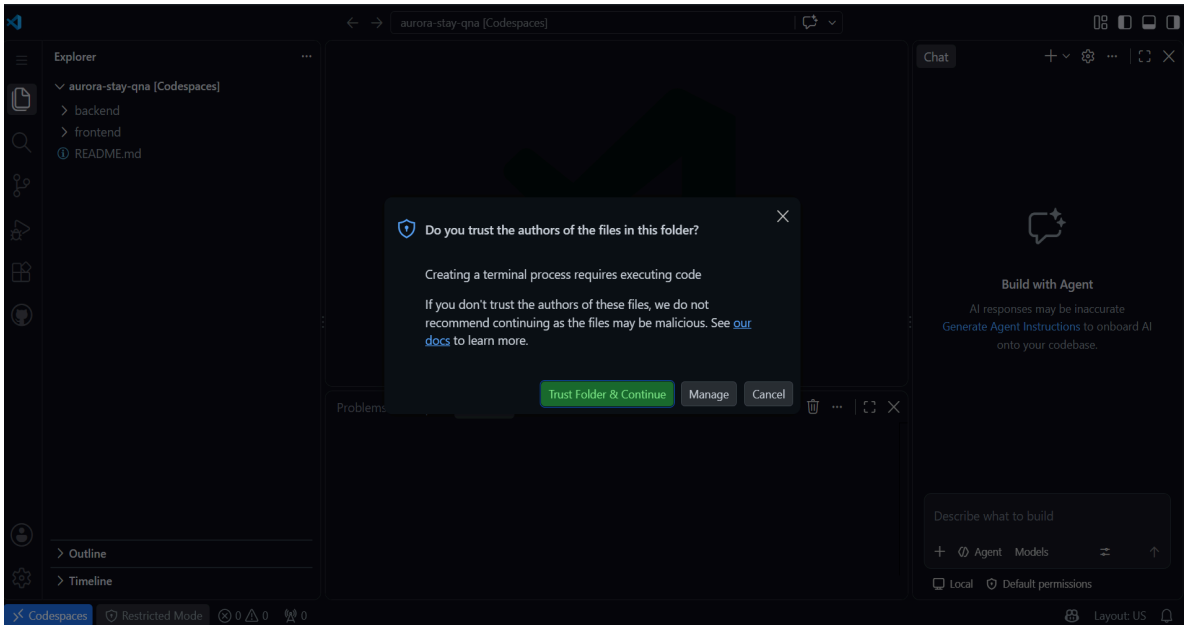


Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

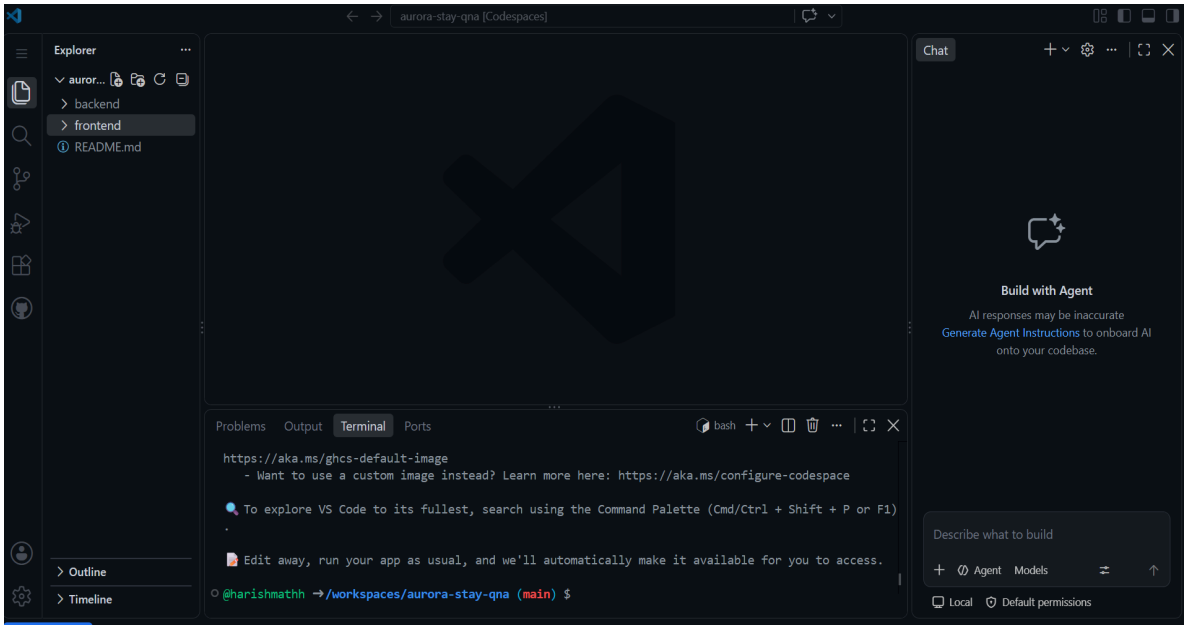
This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces

Step 4: When the Codespace loads for the first time, a dialog box will pop up asking whether you trust the authors of the files in this folder. Click Trust Folder & Continue to proceed. This is required because the Codespace needs to execute code from the repository.



Step 5: After a few moments, the Codespace will finish setting up. You will see a VS Code-like editor in the browser with the project files listed on the left side in the Explorer panel. The Terminal panel will be open at the bottom with a welcome message from GitHub Codespaces.



Section 3: Building the Backend Docker Image

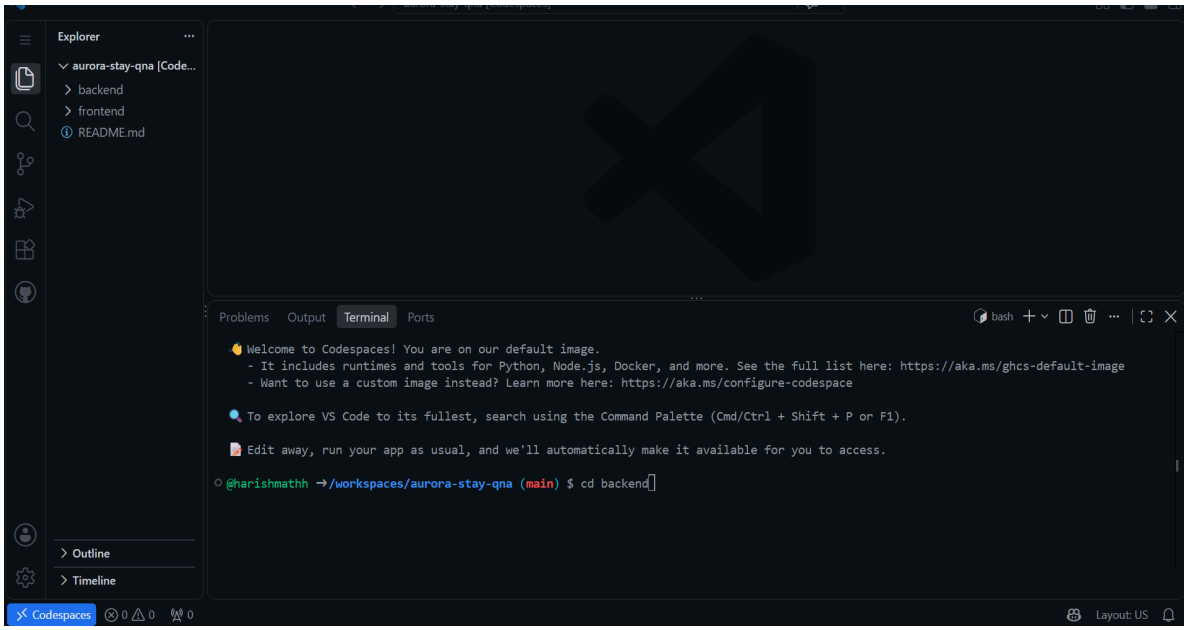
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces

Step 6: In the terminal, navigate to the backend directory by running the following command:

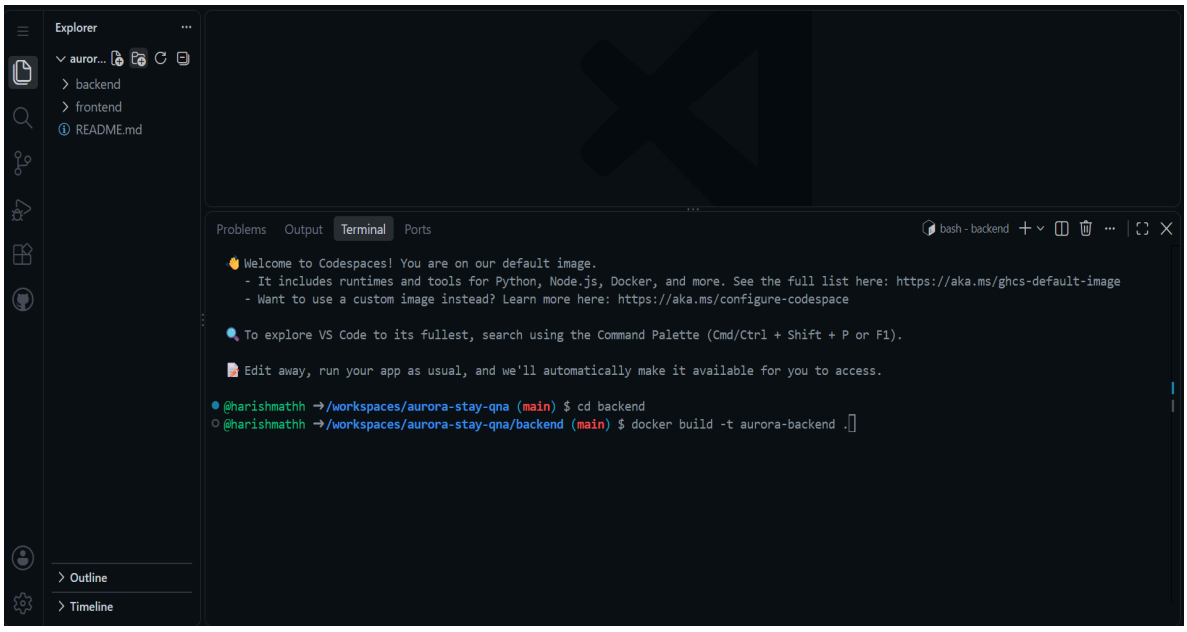
cd backend



Step 7: Once you are inside the backend directory, build the Docker image for the backend service by running:

docker build -t aurora-backend .

This command tells Docker to build an image from the Dockerfile present in the current directory and tag it with the name aurora-backend.

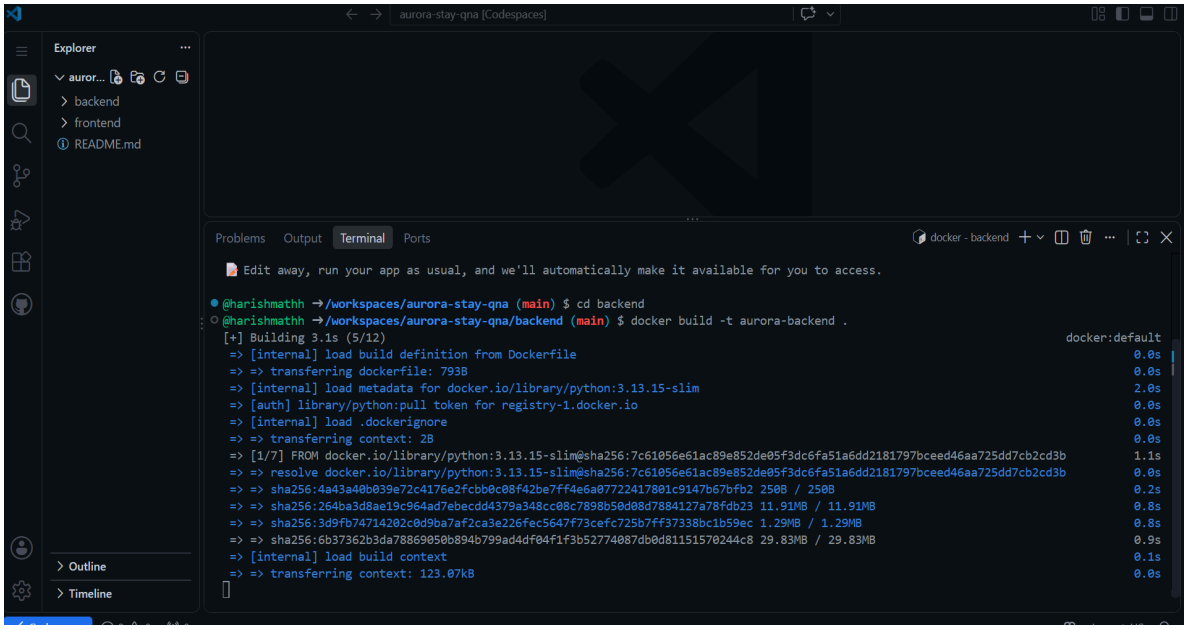


Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces

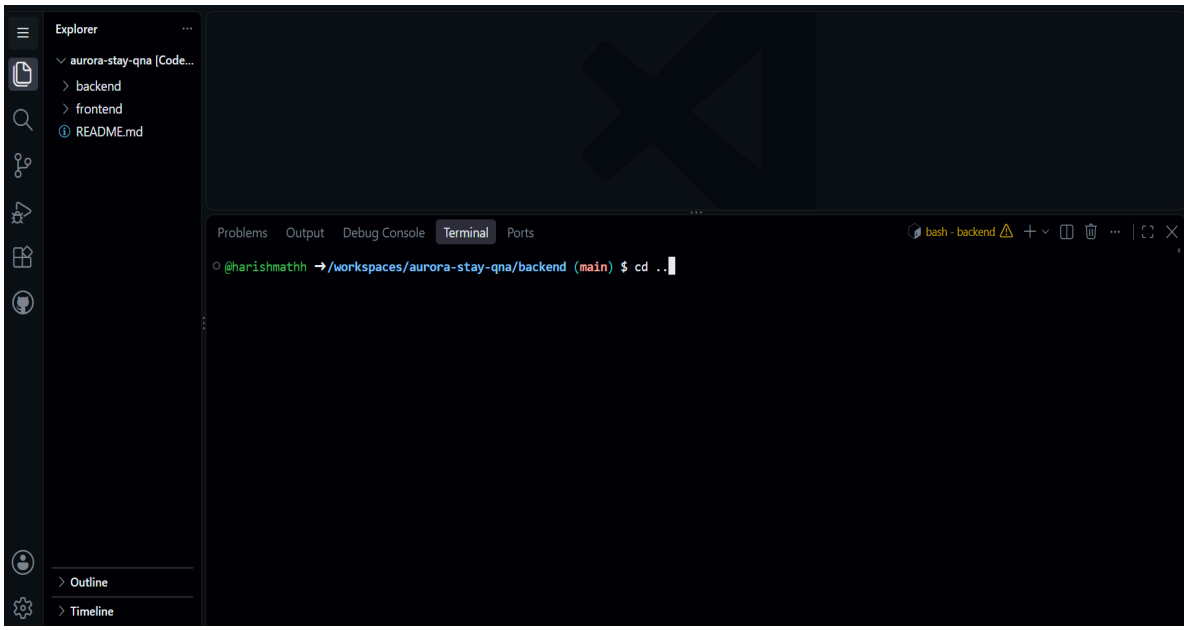
Step 8: Docker will begin pulling the base Python image, installing dependencies, and building the image layer by layer. Wait for the build process to complete. You will see various build steps being executed in the terminal output.



Section 4: Building the Frontend Docker Image

Step 9: After the backend image is built, navigate back to the root of the project by running:

cd ..

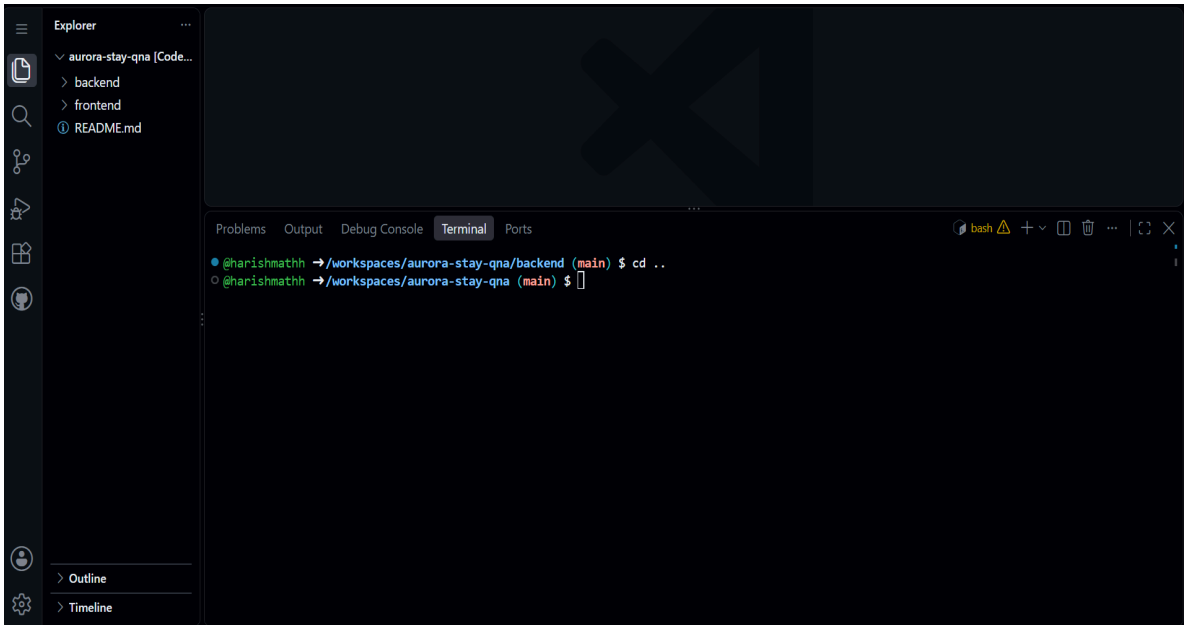


Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

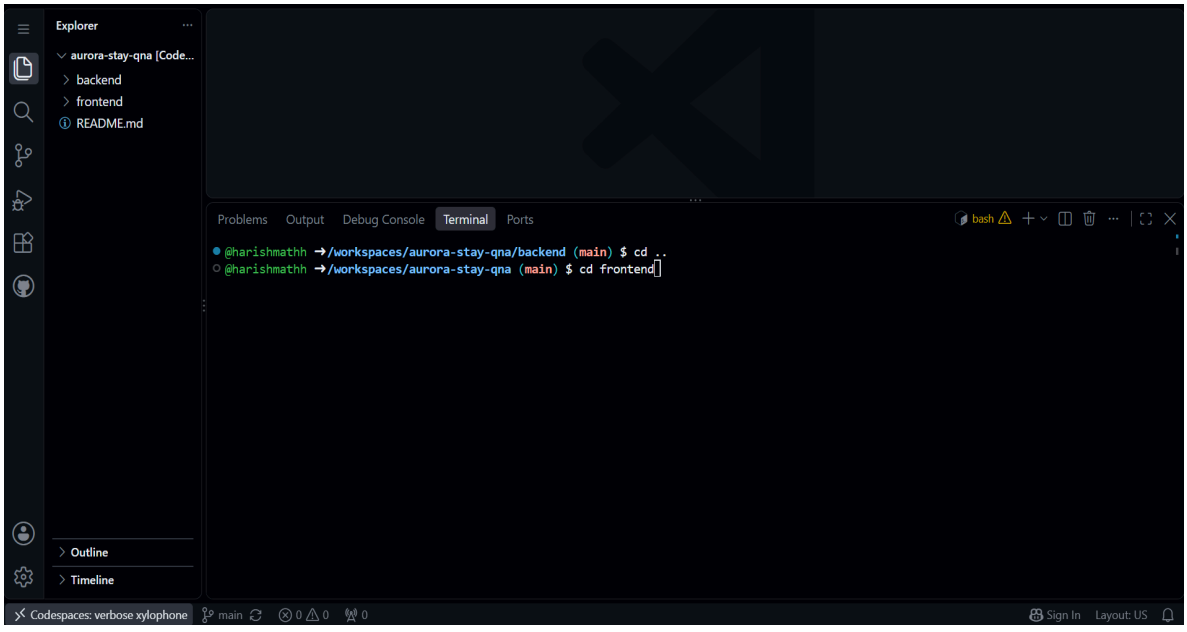
Deploying a RAG Application Using GitHub Codespaces

Step 10: Confirm that you are back in the project root directory. The terminal prompt should show the path ending with aurora-stay-qna or your GitHub repository name.



Step 11: Now navigate into the frontend directory:

cd frontend

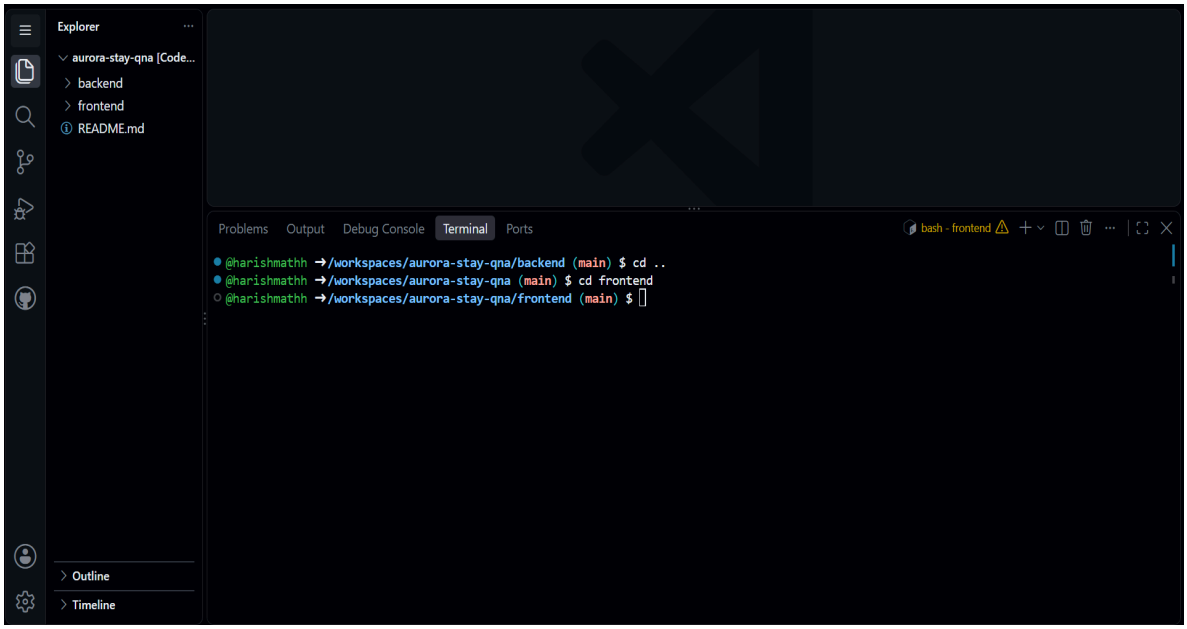


Step 12: Verify that the terminal prompt now shows the path ending with aurora-stay-qna/frontend.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

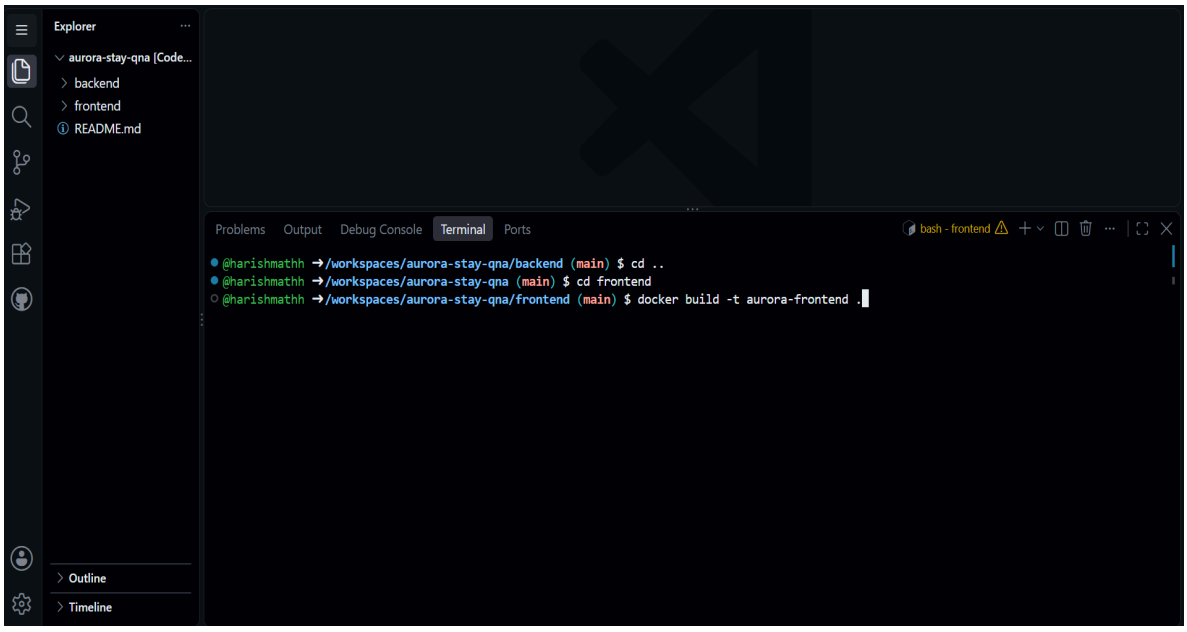
This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces



Step 13: Build the Docker image for the frontend service by running:

docker build -t aurora-frontend .

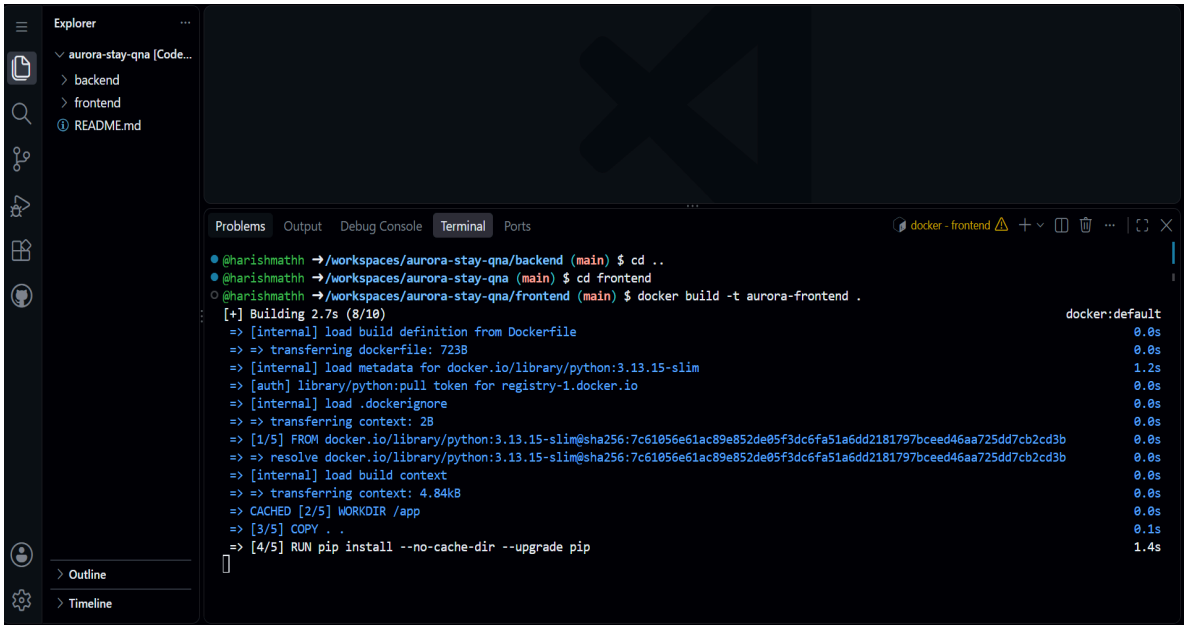


Docker will start building the frontend image. You will see it pulling the Python base image, copying files, upgrading pip, and installing the dependencies listed in requirements.txt.

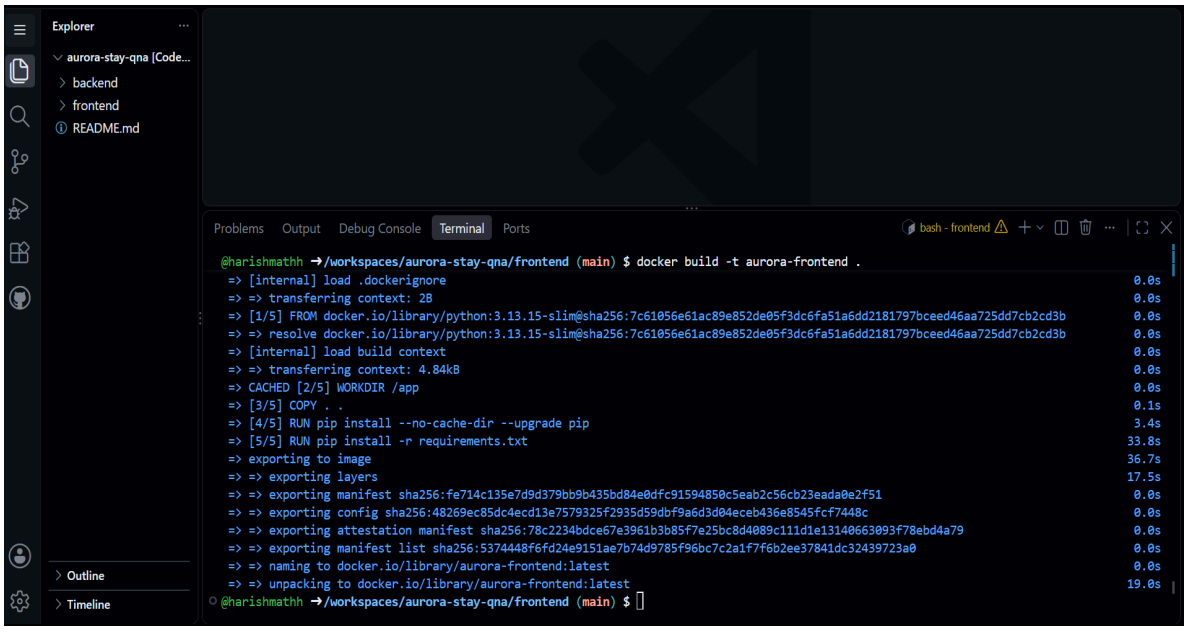
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces



Step 14: Wait for the frontend image build to finish. Once done, the terminal will show the build completion messages and return you to the command prompt.



Section 5: Running the Backend Docker Container

Step 15: Now it is time to run the backend container. Execute the following command in the terminal:

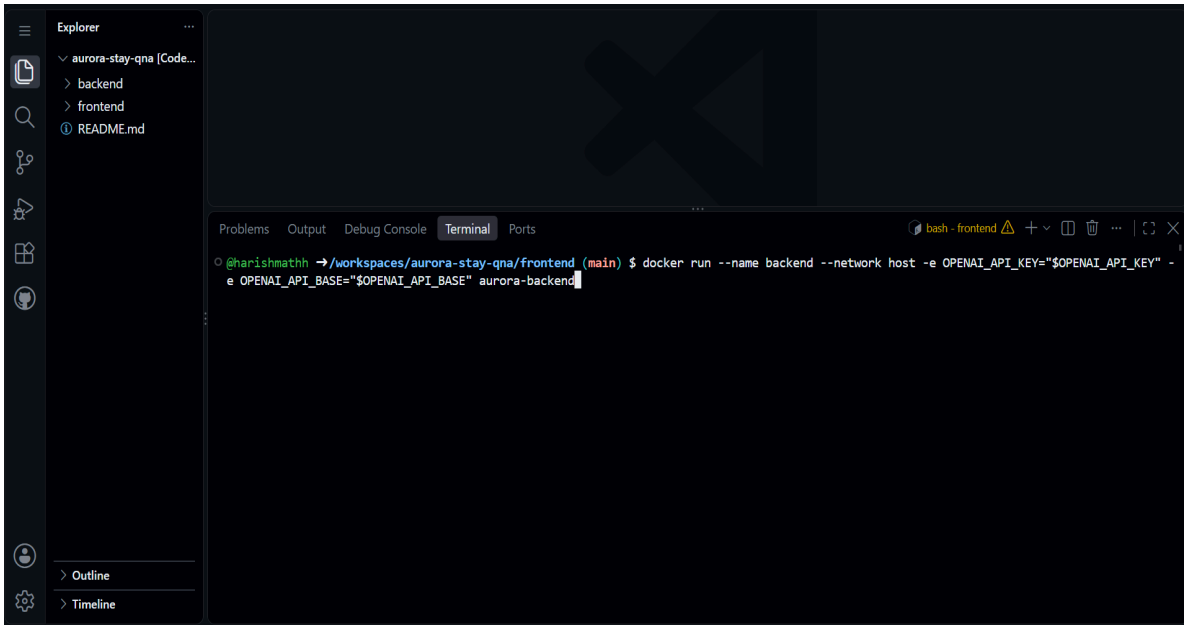
docker run --name backend --network host -e OPENAI_API_KEY="$OPENAI_API_KEY" -e OPENAI_API_BASE="$OPENAI_API_BASE" aurora-backend

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

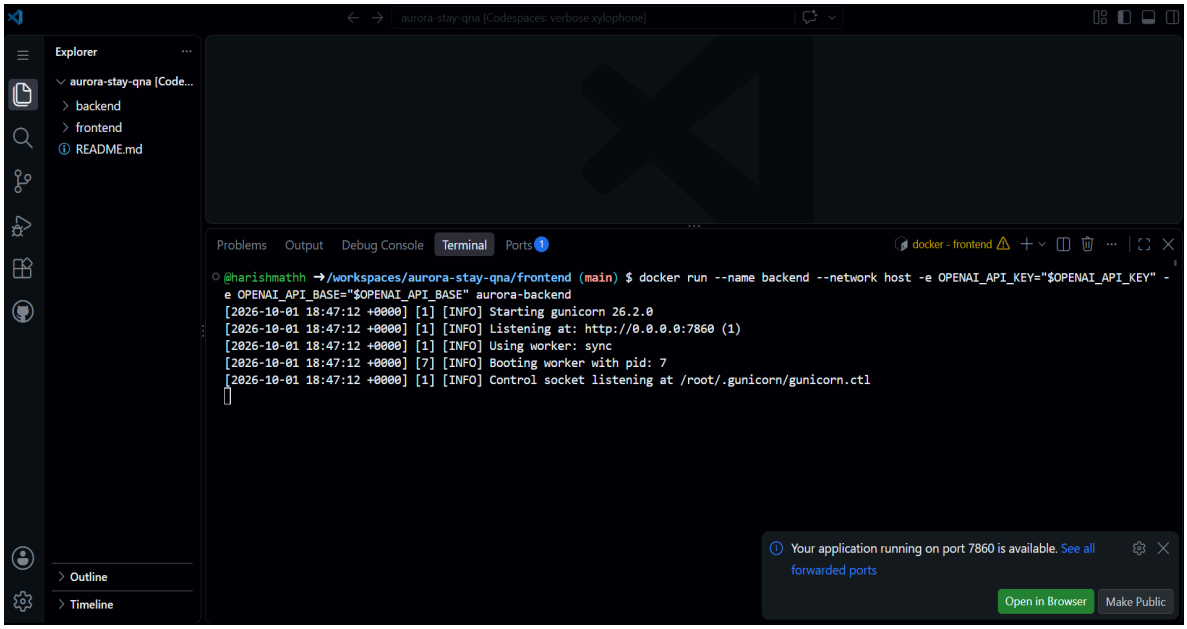
This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces

This command starts a container named "backend" using the host network. It also passes the OpenAI API key and base URL as environment variables, which were previously added as secrets in the Codespace settings.



Step 16: The backend server will start up. You should see Gunicorn logs indicating that it is listening on port 7860. A notification will also appear in the bottom-right corner of the Codespace saying that your application running on port 7860 is available. Click "Make Public" to make the port public so the backend can be accessed from outside of Codespace.

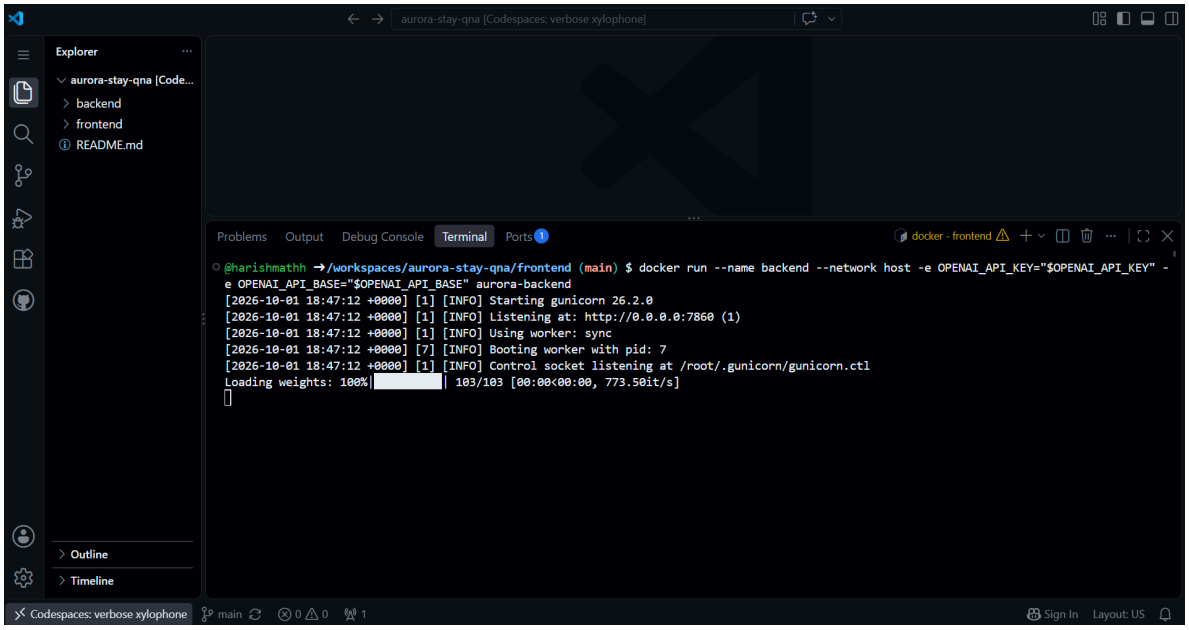


Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

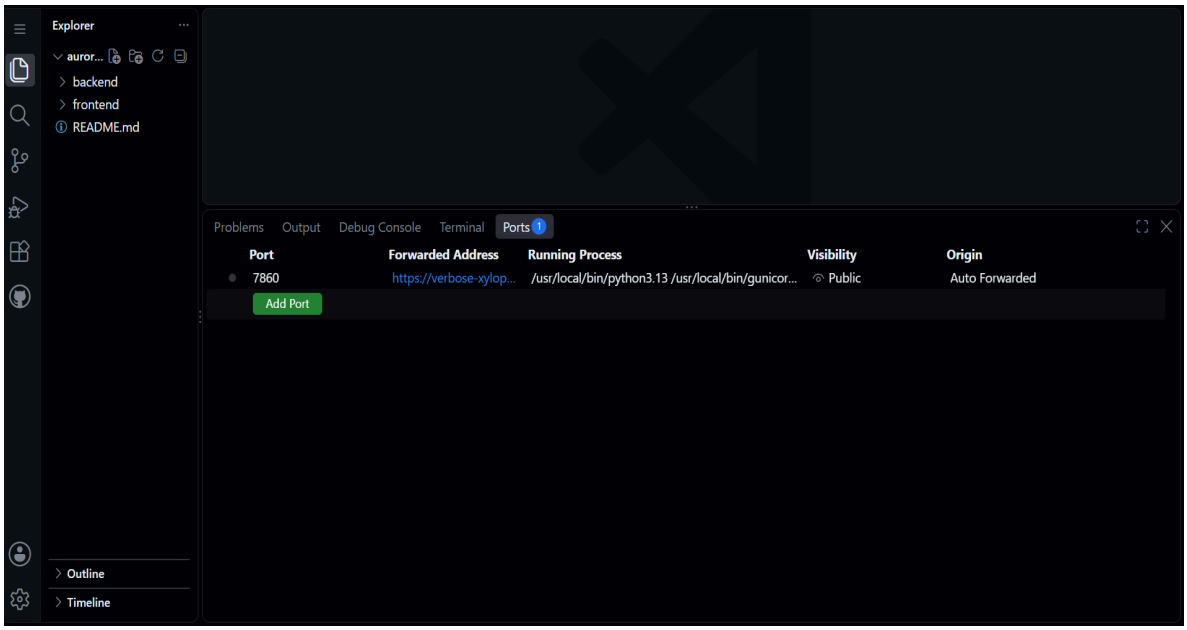
This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces

Step 17: The backend will continue loading the model weights. Wait until the progress bar reaches 100% before proceeding further.



Step 18: Click on the Ports tab in the terminal panel to verify that port 7860 is now listed. It should show the forwarded address, the running process (gunicorn), and the visibility set to Public.

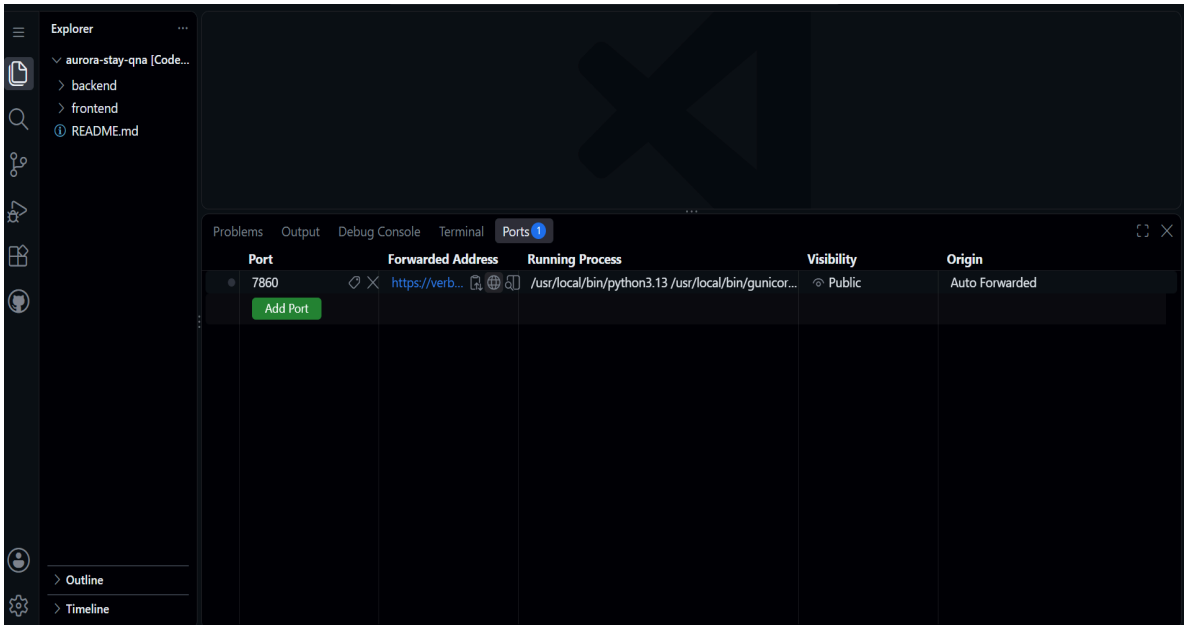


Step 19: Click on the forwarded address link for port 7860 to open the backend API in the browser. You can also hover over the port row to see the full URL and additional options.

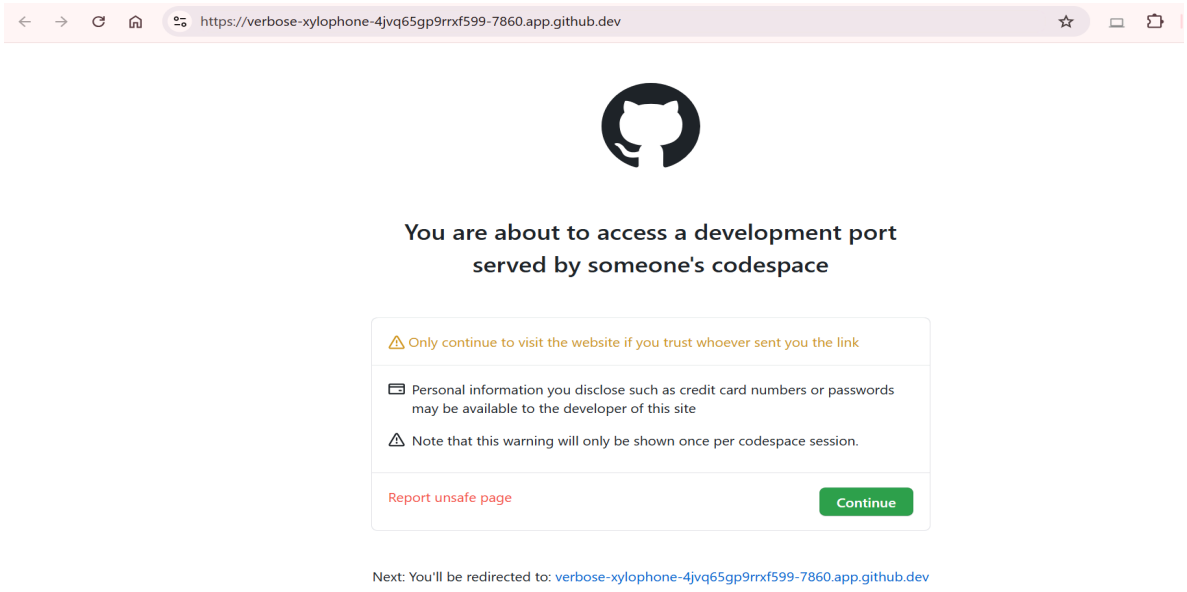
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces



Step 20: A GitHub warning page will appear in the browser, notifying you that you are about to access a development port served by a Codespace. Read the warning and click Continue to proceed.

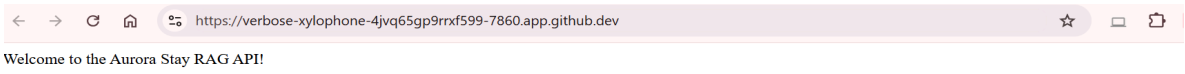


Step 21: After clicking Continue, the browser will load the backend API root endpoint. You should see a message that reads "Welcome to the Aurora Stay RAG API!" confirming that the backend is running correctly. **Copy the URL and save it somewhere, as this URL will be used to access the backend programmatically.**

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

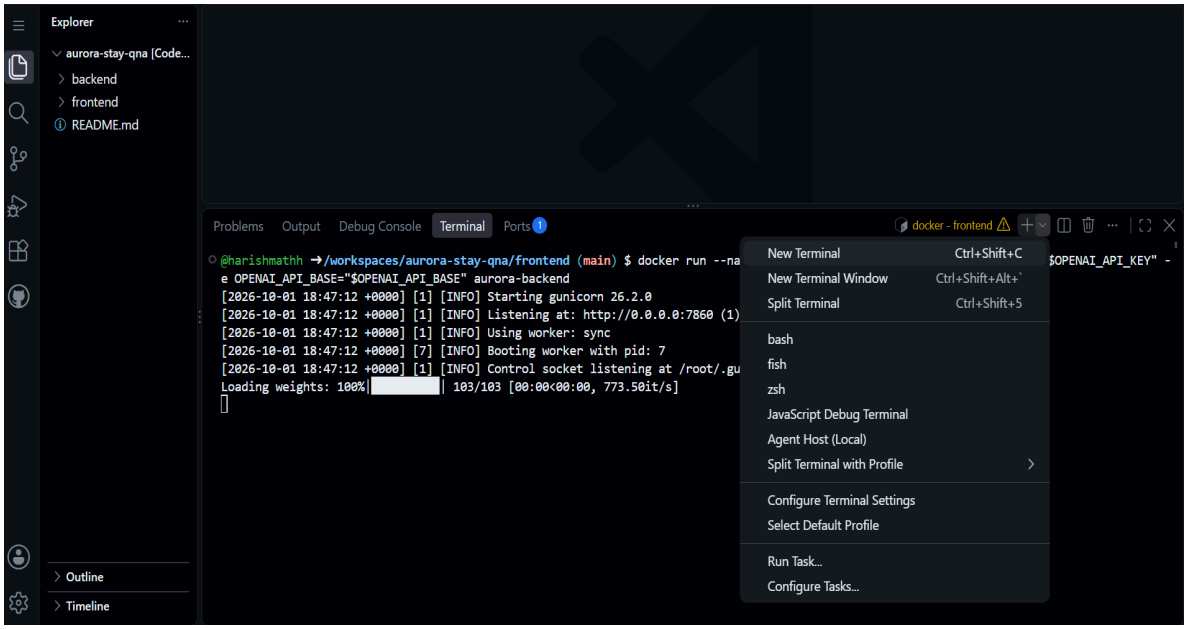
This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces



Section 6: Running the Frontend Docker Container

Step 22: Since the backend terminal is occupied by the running container, you need to open a new terminal. Click the dropdown arrow next to the "+" icon in the terminal panel and select bash (or click New Terminal from the context menu).

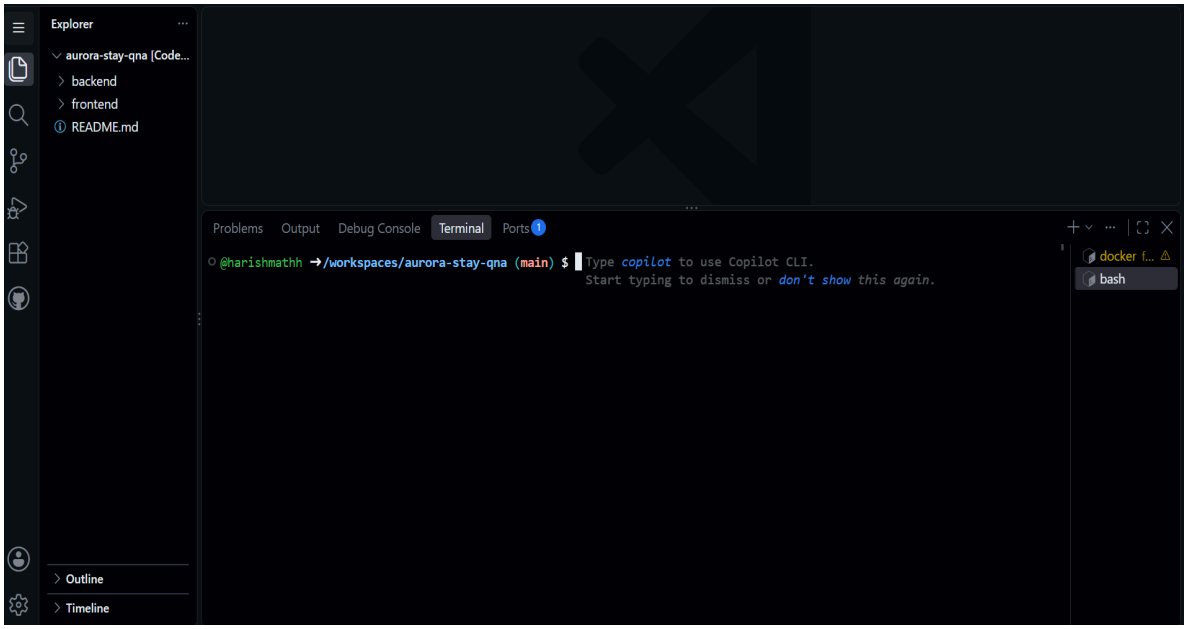


Step 23: A new bash terminal tab will open alongside the existing docker terminal. You will see the new prompt ready for input in the project root directory.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

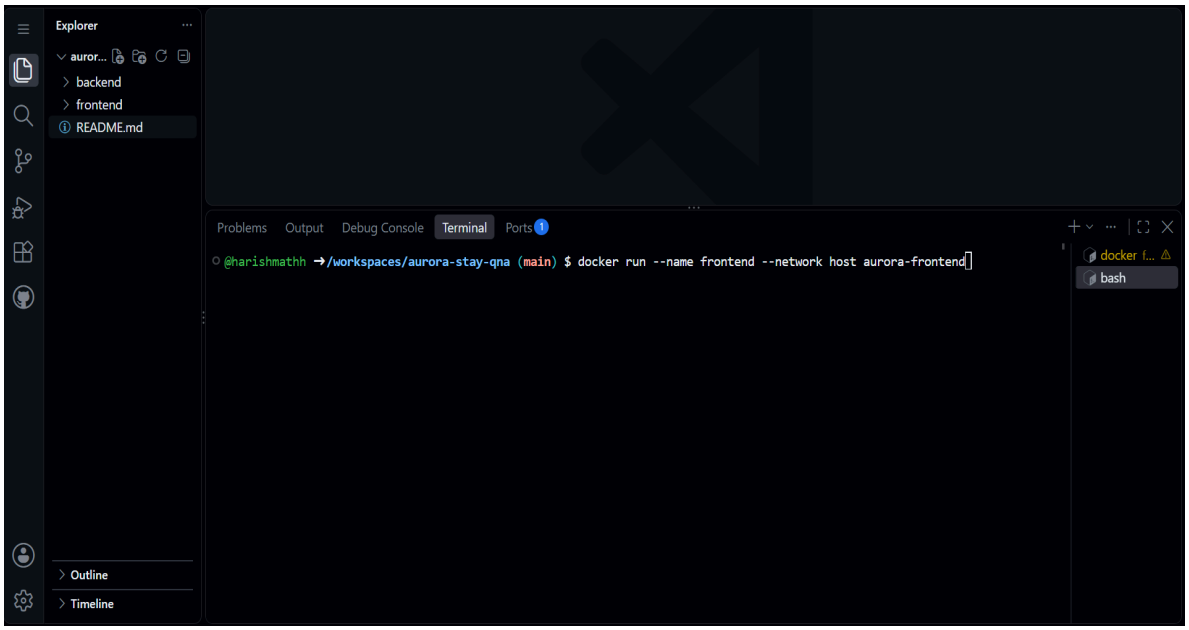
This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces



Step 24: In the new terminal, run the following command to start the frontend container:

docker run --name frontend --network host aurora-frontend

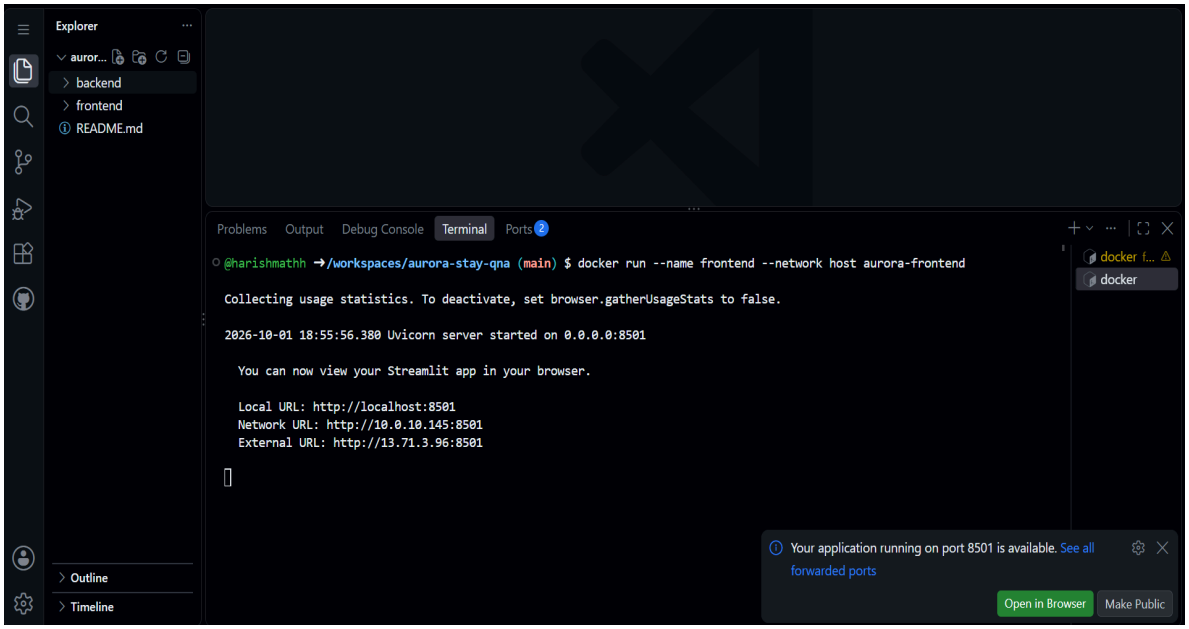


Step 25: The frontend container will start up. The terminal will show the Uvicorn server starting on port 8501 along with a Streamlit message indicating the local, network, and external URLs. A notification will appear at the bottom-right corner saying the application on port 8501 is available.

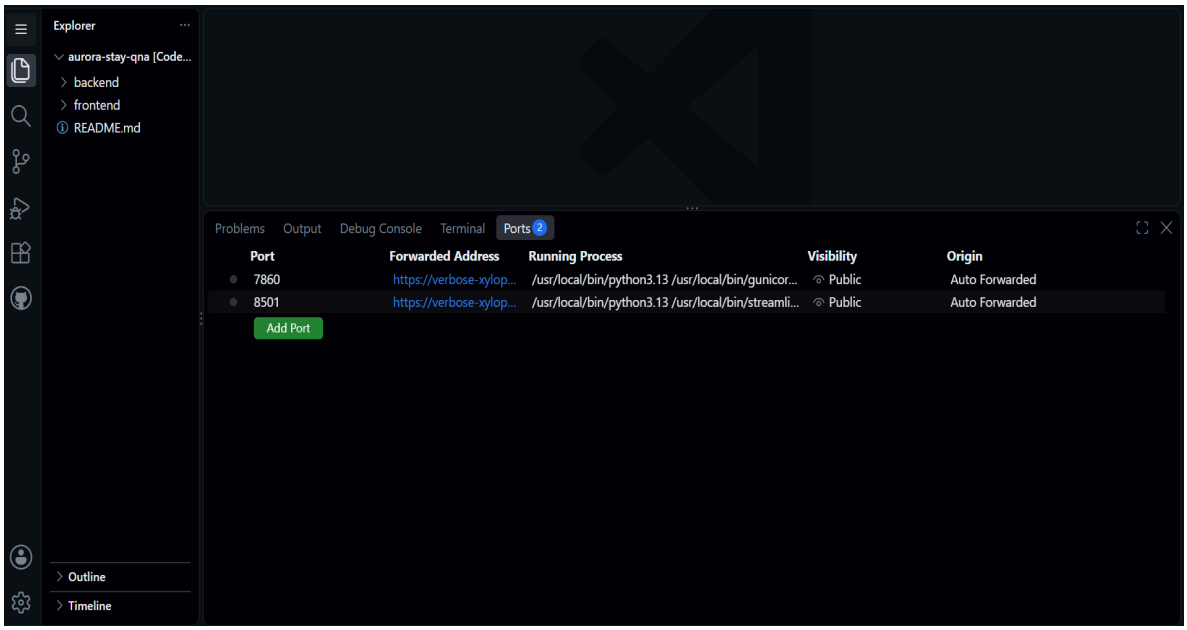
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces



Step 26: Switch to the Ports tab again. You should now see two ports listed: port 7860 for the backend (gunicorn) and port 8501 for the frontend (streamlit). Both should have their visibility set to Public.

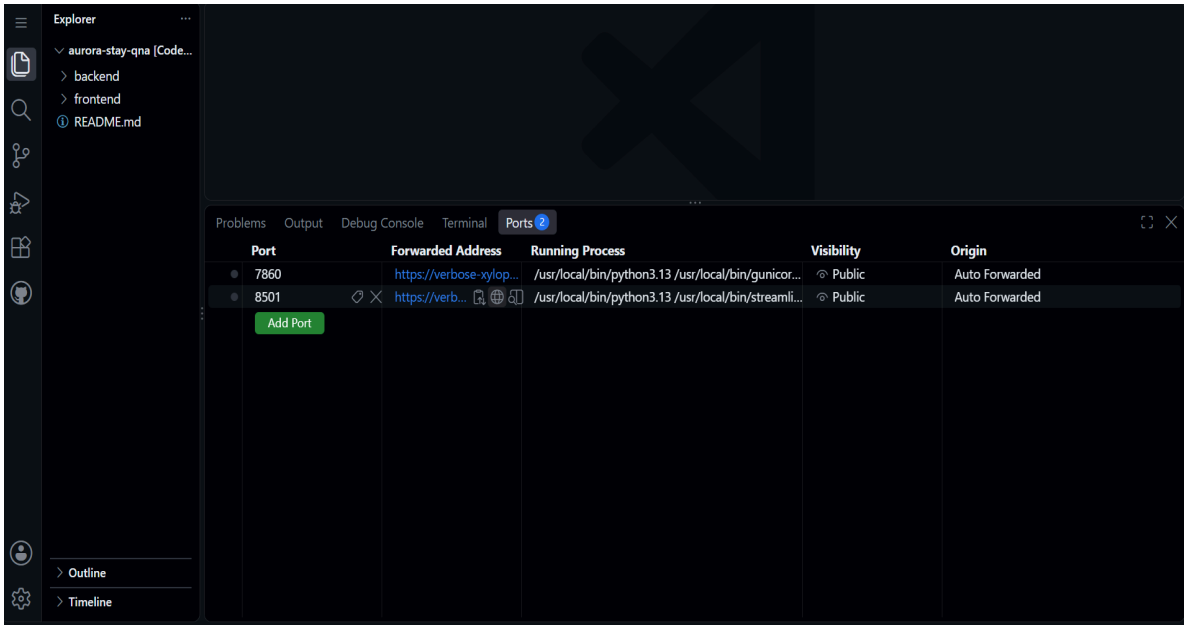


Step 27: Click on the forwarded address for port 8501 to open the frontend application in the browser.

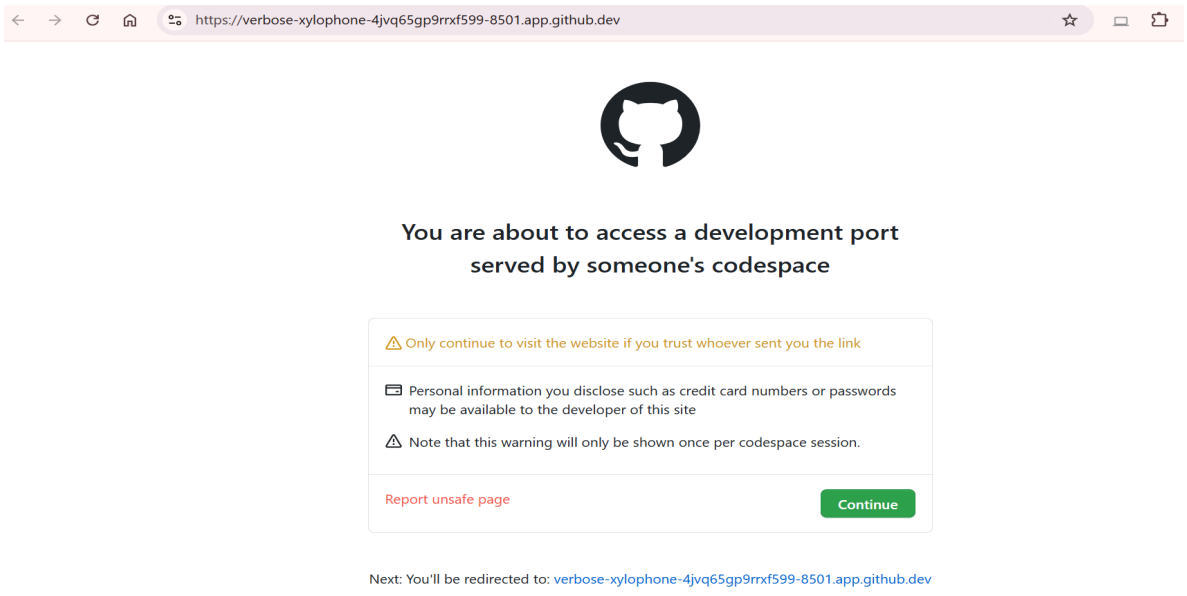
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces



Step 28: The same GitHub warning page will appear for the frontend port. Click Continue to proceed.



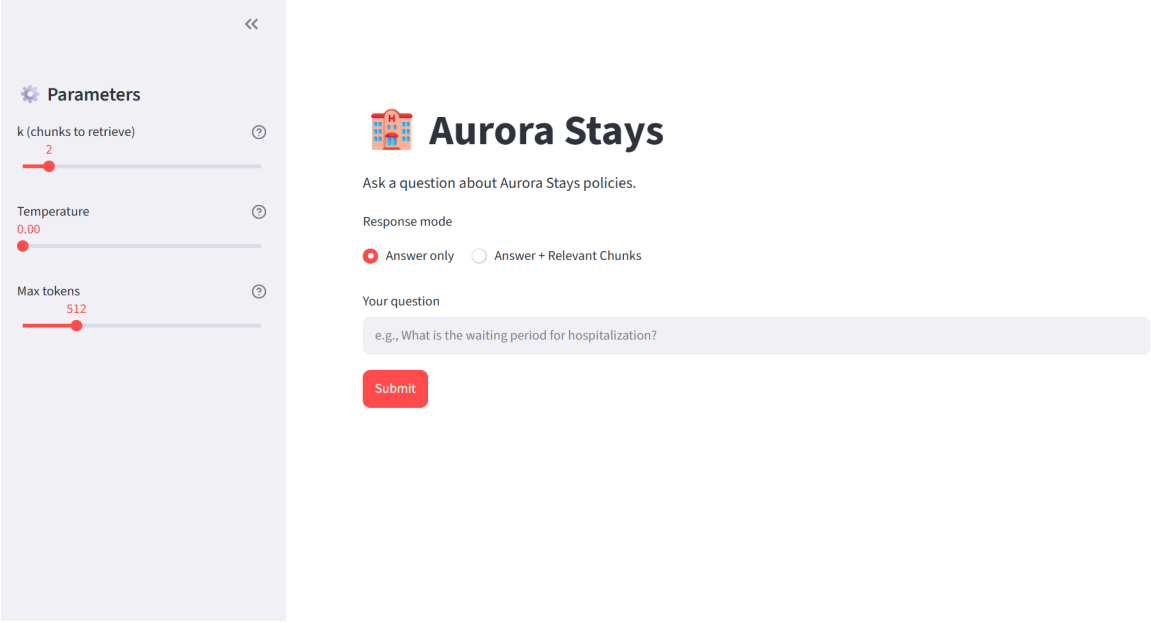
Section 7: Testing the Deployed RAG Application

Step 29: After clicking Continue, the Aurora Stays frontend application will load in the browser. You will see a Streamlit-based interface with a sidebar containing parameters (such as k chunks to retrieve, Temperature, and Max tokens) and a main panel with a text input field where you can type your question. The response mode is set to "Answer only" by default.

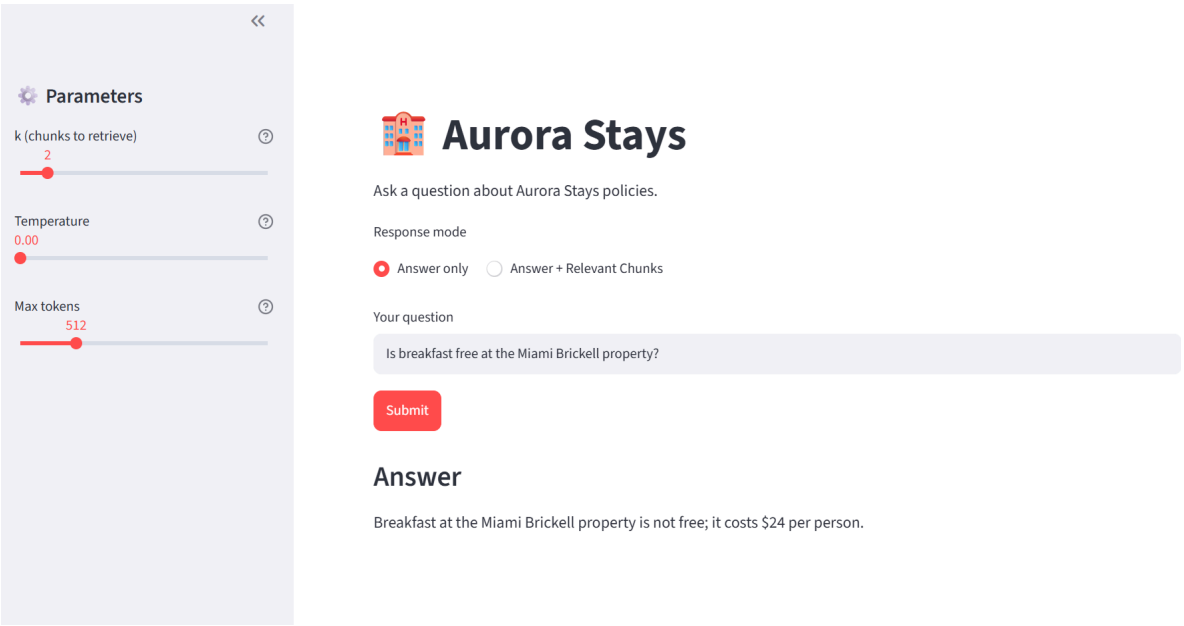
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces



Step 30: Type a question in the input field, for example: "Is breakfast free at the Miami Brickell property?" and click Submit. The application will process the query and display the answer below. In this case, the answer states that breakfast at the Miami Brickell property is not free and costs $24 per person.

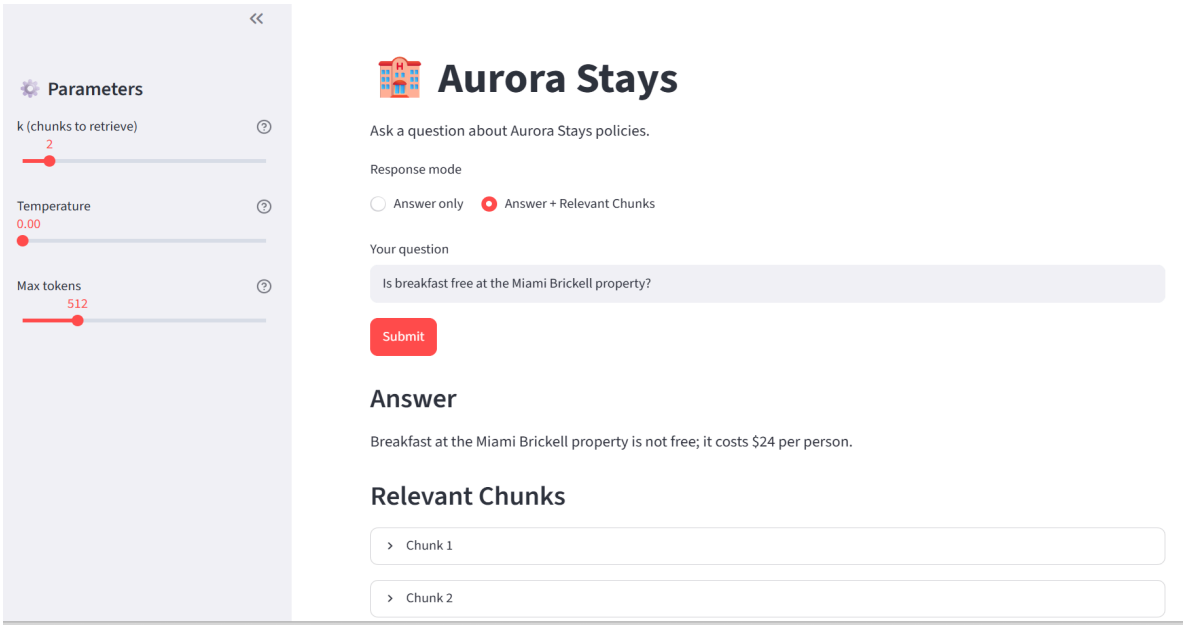


Step 31: To see the retrieved context alongside the answer, switch the response mode to "Answer + Relevant Chunks" and submit the same question again. The application will now display the answer followed by expandable sections labeled Chunk 1, Chunk 2, and so on.

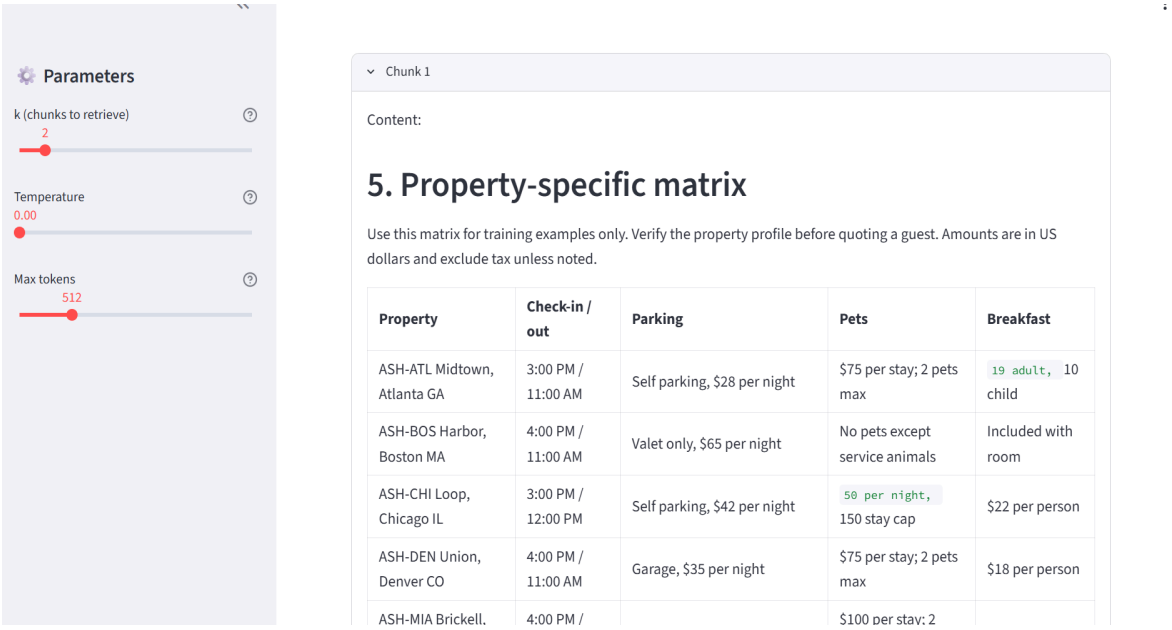
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces



Step 32: Click on any chunk to expand it and view the raw content that was retrieved from the knowledge base. In this example, Chunk 1 shows a property-specific matrix with details about check-in/check-out times, parking costs, pet policies, and breakfast pricing across various Aurora Stays properties.



Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Deploying a RAG Application Using GitHub Codespaces

This completes the deployment and testing of the Aurora Stays RAG application using GitHub Codespaces. Both the backend API and the Streamlit frontend are now running as Docker containers inside the Codespace, accessible through the forwarded ports.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

This file is meant for personal use by tirumalabpc@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.